Universidad Francisco de Paula Santander
Ocaña - Colombia
Vigilada Mineducación

Ingeniería de Sistemas

Sistema de Gestión de Inventario Postobón Ocaña

Documento de especificación del diseño e implementación de la Base de Datos

Autores

Dairo Julián Parada Reyes – 0192288
Yiceth Yuliana Velásquez Gómez – 0192312

Versión 2.0 — Noviembre de 2026

Control de cambios

Fecha	Versión	Descripción	Autor
23/08/2026	1.1	Estrategia y análisis: negocio, misión, requisitos, reglas y supuestos iniciales.	Equipo del proyecto
27/10/2026	1.5	Implementación en PostgreSQL: corrección del modelo físico, DDL, triggers y carga de datos.	Equipo del proyecto
05/11/2026	2.0	Documento completo: modelos ER, especificaciones, C4, pruebas UAT, despliegue y Anexo 1 (IA).	Equipo del proyecto

Tabla de contenido

1. Introducción..... 5
2. Estrategia y Análisis..... 7
2.1 Negocio y misión..... 7
2.2 Requisitos de información..... 7
2.3 Reglas del negocio..... 8
2.4 Supuestos..... 9
2.5 Modelo conceptual (Diagrama de ER)..... 10
2.6 Modelo lógico (Diagrama de ER)..... 11
Normalización..... 12
Análisis CRUD..... 12
Refinamiento del modelo lógico..... 13
3. Diseño..... 14
3.1 Especificaciones de diseño..... 14
3.2 Transformación del modelo lógico a diagrama de tablas e instancias..... 14
4. Creación..... 20
4.1 Modelo físico..... 20
4.2 Script DDL y DML..... 21
Triggers..... 22
Carga de datos (DML)..... 22
4.3 Modelo C4..... 23
Nivel 1: Diagrama de contexto del sistema..... 24
Nivel 2: Diagrama de contenedores..... 25
4.4 Repositorio GitHub..... 25
5. Transición..... 27
5.1 Pruebas de aceptación de usuario (UAT)..... 27
Estrategia de pruebas..... 27
Pruebas de integridad de datos..... 28
Pruebas de consultas y reportes..... 29
Pruebas de concurrencia..... 29
Pruebas de seguridad..... 30
Pruebas de rendimiento..... 30
Pruebas de recuperación ante fallos..... 31
Criterios de éxito y aprobación..... 31
5.2 Matriz de trazabilidad..... 31

6. Producción..... 33

6.1 Desplegar el modelo físico en la nube (Supabase)..... 33

Problemas encontrados en el despliegue y su solución..... 34

6.2 Colocar en funcionamiento el sistema..... 34

7. Conclusión..... 36

8. Recomendaciones..... 37

9. Glosario..... 38

10. Bibliografía..... 39

Anexo 1. Instrumento de recolección de evidencias..... 41

E1. Diseño relacional..... 41

E2. Prompt inicial..... 41

E3. Excel inicial..... 42

E4. Registro de revisión..... 42

E5. Historial de interacción..... 43

E6. Excel definitivo..... 44

E7. Base de datos poblada..... 44

E8. Validación SQL..... 45

E9. Reflexión individual..... 46

1. Introducción

Postobón S.A. es una empresa colombiana dedicada a la producción, distribución y comercialización de bebidas: gaseosas, jugos, aguas, tés, hidratantes y energizantes. Opera plantas de producción y centros de distribución en distintas ciudades del país; uno de ellos atiende la provincia de Ocaña, en Norte de Santander.

En un centro de distribución el inventario cambia todos los días: llegan pedidos de los proveedores, se reciben lotes con fecha de vencimiento y hay que saber qué productos están por debajo de su stock mínimo. Cuando esta información se lleva en hojas de cálculo aparecen datos duplicados, totales que no cuadran y lotes que no se sabe de qué pedido vinieron.

Este documento especifica el diseño y la implementación de una base de datos relacional para el **módulo de gestión de inventario del centro de distribución Postobón Ocaña**. Sigue las etapas de la asignatura: estrategia y análisis, diseño, creación, transición y producción. El resultado es un esquema PostgreSQL de 8 tablas de negocio y 1 de auditoría, protegido con llaves foráneas sin eliminación en cascada, restricciones CHECK y 29 triggers, poblado con datos sintéticos generados con apoyo de inteligencia artificial y verificado con 62 pruebas de ataque y las pruebas de aceptación de la sección 5.

El modelo relacional se fundamenta en Codd (1970) y el modelado entidad-relación en Chen (1976); las decisiones de normalización siguen a Elmasri y Navathe (2016) y a Silberschatz et al. (2020); la implementación sigue la documentación oficial de PostgreSQL (PostgreSQL Global Development Group [PGDG], 2024a) y el material de clase de la asignatura (Cuesta Quintero, 2026).

Tabla 1

Datos generales del proyecto

Elemento	Descripción
Asignatura	Bases de datos relacionales — Docente Byron Cuesta Quintero
Institución	Universidad Francisco de Paula Santander Ocaña (UFPSO), Ingeniería de Sistemas
Integrantes	Dairo Julián Parada Reyes (0192288) Yiceth Yuliana Velásquez Gómez (0192312)
Sistema gestor	PostgreSQL 16 (administrado con pgAdmin 4); despliegue en Supabase
Base de datos / esquema	`postobon` / `inventario_postobon`
Alcance	Catálogo de productos y categorías, proveedores, empleados, órdenes de abastecimiento, lotes recibidos, existencias vigentes y auditoría de cambios

Elemento	Descripción
Fuera de alcance	Varias bodegas, despachos (salidas), devoluciones, facturación e integración automática con otros sistemas

2. Estrategia y Análisis

2.1 Negocio y misión

Negocio. El centro de distribución de Postobón en Ocaña recibe bebidas de plantas y proveedores, las almacena y las despacha a tiendas y distribuidores de la provincia. Para abastecerse emite órdenes de abastecimiento a sus proveedores; cada orden llega en uno o varios lotes que el personal de bodega recibe, cuenta y registra con su fecha de vencimiento.

Misión del sistema. Diseñar e implementar una base de datos relacional que soporte el módulo de inventario del centro de distribución Postobón Ocaña, de forma que el equipo pueda registrar el catálogo, controlar el ciclo de vida de las órdenes de abastecimiento, saber qué llegó en cada lote y quién lo recibió, conocer la existencia vigente de cada producto frente a su stock mínimo y máximo, y conservar un historial inalterable de los cambios.

2.2 Requisitos de información

Los requisitos de información describen los datos que el sistema debe almacenar y las preguntas que debe poder responder. Se identificaron con el equipo a partir del proceso de abastecimiento descrito en la sección 2.1 y se refinaron al acotar el alcance a un solo centro de distribución (ver sección 2.6, refinamiento).

Tabla 2

Requisitos de información

ID	Requisito	Tablas que lo soportan
RI-01	Mantener el catálogo de productos con su categoría, unidad de medida, precio de referencia y stock mínimo y máximo.	category, product
RI-02	Identificar los productos cuya existencia vigente está por debajo del stock mínimo o por encima del máximo.	product, batch, batch_detail (vista v_existencias)
RI-03	Registrar los proveedores con NIT único y dirección.	supplier
RI-04	Registrar las órdenes de abastecimiento con proveedor, fecha de emisión, estado y valor total.	supply_order, order_detail
RI-05	Conservar el historial de compras realizadas a cada proveedor.	supplier, supply_order, order_detail
RI-06	Registrar los lotes recibidos por cada orden, con su fecha de vencimiento, cantidad y empleado responsable.	batch, batch_detail, employee
RI-07	Identificar qué proveedor suministró un lote determinado.	batch, supply_order, supplier

ID	Requisito	Tablas que lo soportan
RI-08	Identificar los productos y lotes próximos a vencer.	batch, batch_detail, product (vista v_lotes_por_vencer)
RI-09	Identificar el empleado que recibió un lote y el usuario de base de datos que hizo cada cambio.	batch, employee, audit_log
RI-10	Medir el cumplimiento de cada proveedor: unidades pedidas frente a unidades recibidas.	order_detail, batch_detail (vista v_cumplimiento_proveedor)
RI-11	Consultar el valor de las compras por estado, proveedor y periodo.	supply_order

A partir de los requisitos se identificaron ocho entidades: **Categoría**, **Producto**, **Proveedor**, **Empleado**, **Orden de abastecimiento**, **Detalle de orden**, **Lote** y **Detalle de lote**. Se agregó una tabla técnica de **Auditoría**, que no es una entidad del negocio sino un mecanismo de control.

2.3 Reglas del negocio

Las reglas del negocio fijan las condiciones que los datos deben cumplir. Cada regla tiene un mecanismo de implementación verificable en la base de datos, lo que permite seguirla hasta las pruebas de la sección 5 (ver Tabla de trazabilidad, sección 5.2).

Tabla 3

Reglas del negocio

ID	Regla	Tipo	Implementación
RN-01	Todo producto pertenece a una única categoría.	E	FK product_category_fk, NOT NULL
RN-02	El precio de referencia es mayor que cero; el stock mínimo es ≥ 0 y el máximo es mayor que el mínimo.	P	CHECK product_price_ck, min_ck, max_ck
RN-03	La unidad de medida es una de: UNIDAD, PACA, CANASTA, CAJA, SIXPACK, FARDO o BOLSA.	P	CHECK product_unit_ck
RN-04	El nombre de producto y de categoría no se repite, sin importar mayúsculas o espacios.	P	Índice único upper(name) + trigger T1
RN-05	Todo proveedor tiene un NIT único con formato #########-#.	E	UNIQUE + CHECK supplier_tax_id_ck
RN-06	Todo empleado tiene un cargo de la lista oficial y un usuario único en minúsculas.	E	CHECK employee_position_ck, username_ck, UNIQUE
RN-07	Toda orden pertenece a un proveedor, nace PENDIENTE y su fecha de emisión no es futura.	E	FK + trigger T2
RN-08	Estados válidos: PENDIENTE → APROBADA o CANCELADA; APROBADA → RECIBIDA o CANCELADA. RECIBIDA y CANCELADA son finales.	P	Trigger T2
RN-09	Una orden se aprueba solo si tiene productos, se marca	P	Trigger T2

ID	Regla	Tipo	Implementación
	RECIBIDA solo si tiene lotes y no se cancela si ya recibió lotes.		
RN-10	El detalle de una orden solo se modifica mientras la orden está PENDIENTE.	P	Trigger T3
RN-11	El total de la orden es la suma de cantidad × precio de su detalle y no se digita a mano.	P	Triggers T2 y T4
RN-12	Solo se reciben lotes de órdenes APROBADAS; el vencimiento es posterior a la orden y no supera 5 años.	P	Trigger T5
RN-13	Todo lote es recibido y registrado por un empleado.	E	FK batch_employee_fk
RN-14	No se recibe un producto que no se pidió, ni más de lo pedido, ni más de la cantidad inicial del lote.	P	Trigger T6
RN-15	No se elimina en cascada: un registro con datos relacionados no se puede borrar.	P	ON DELETE RESTRICT + trigger T9 (TRUNCATE)
RN-16	Todo cambio en catálogos y órdenes queda auditado, y la auditoría no se puede modificar ni borrar.	P	Triggers T7 y T8

Nota. E = estructural (define qué información se guarda y cómo se relaciona). P = de procedimiento (define condiciones o acciones que se deben cumplir durante la operación).

2.4 Supuestos

Los supuestos son condiciones que el equipo dio por válidas cuando no fue posible confirmarlas con el negocio. Se mantienen al mínimo y se documentan para que cualquier cambio futuro se pueda rastrear.

Tabla 4

Supuestos del modelo

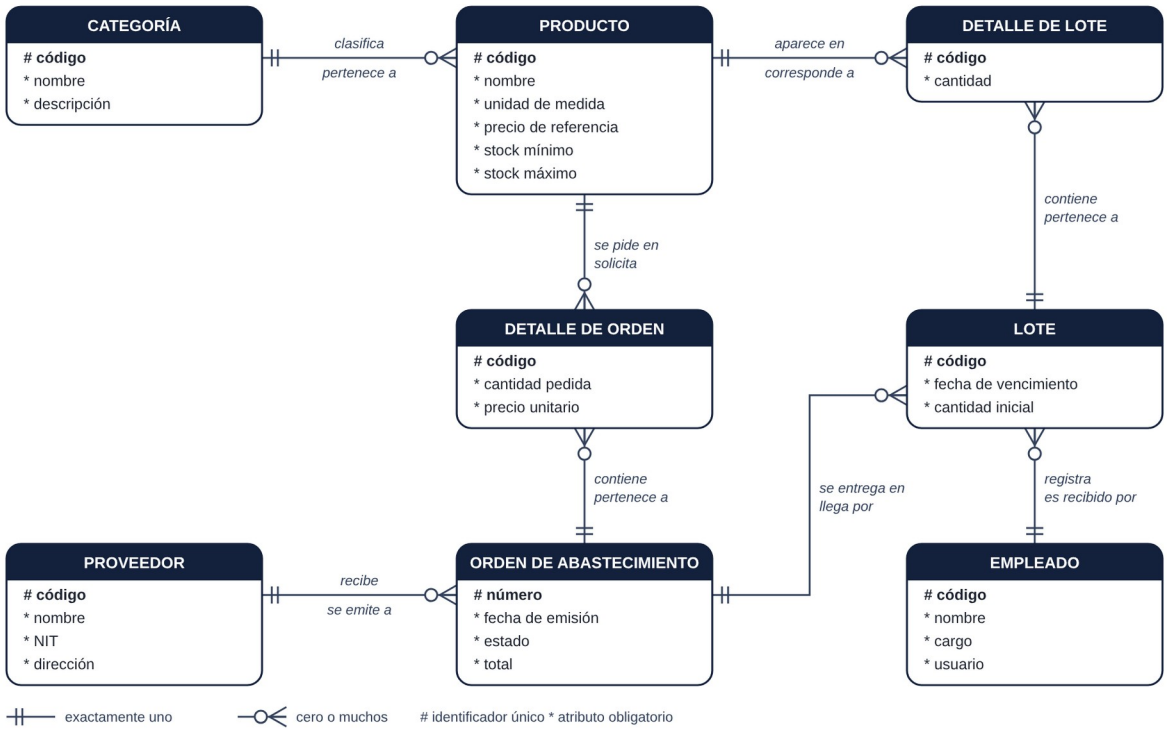
ID	Supuesto
S-01	El sistema atiende un solo centro de distribución (Ocaña). No se modelan varias bodegas ni ubicaciones internas como pasillos o estantes.
S-02	El precio real lo maneja el sistema de ventas de Postobón. El inventario guarda un precio de referencia y el precio unitario negociado en cada orden.
S-03	No se manejan devoluciones ni despachos (salidas). La existencia vigente se calcula como lo recibido en lotes que no han vencido.
S-04	Todas las bebidas tienen fecha de vencimiento; por eso el vencimiento es obligatorio en todos los lotes.
S-05	Un mismo empleado puede recibir lotes de cualquier orden y proveedor.
S-06	Una orden puede llegar en varios lotes, y un lote puede traer varios productos de la misma orden.
S-07	Todos los valores monetarios se expresan en pesos colombianos (COP). No se manejan otras monedas.
S-08	Los datos cargados son sintéticos: tienen la estructura y las reglas reales, pero no corresponden a operaciones reales de Postobón.

2.5 Modelo conceptual (Diagrama de ER)

El modelo conceptual representa las entidades del negocio, sus atributos principales y las relaciones con su cardinalidad, sin detalles de implementación (Chen, 1976; Elmasri y Navathe, 2016). Se usa notación de pata de gallo: dos rayas indican “exactamente uno” y el círculo con la pata indica “cero o muchos”.

Figura 1

Modelo conceptual del inventario del centro de distribución



Nota. Elaboración propia. # identifica la entidad; * indica atributo obligatorio. Cada relación se lee en los dos sentidos, por ejemplo: una categoría clasifica cero o muchos productos; un producto pertenece a exactamente una categoría.

Relaciones identificadas. Una categoría clasifica muchos productos. Un proveedor recibe muchas órdenes de abastecimiento. Una orden contiene muchos detalles, y cada detalle solicita un producto; así se resuelve la relación muchos a muchos entre orden y producto. Una orden se entrega en uno o varios lotes, cada lote es recibido por un empleado y contiene detalles de lote que corresponden a un producto; así se resuelve la relación muchos a muchos entre lote y producto.

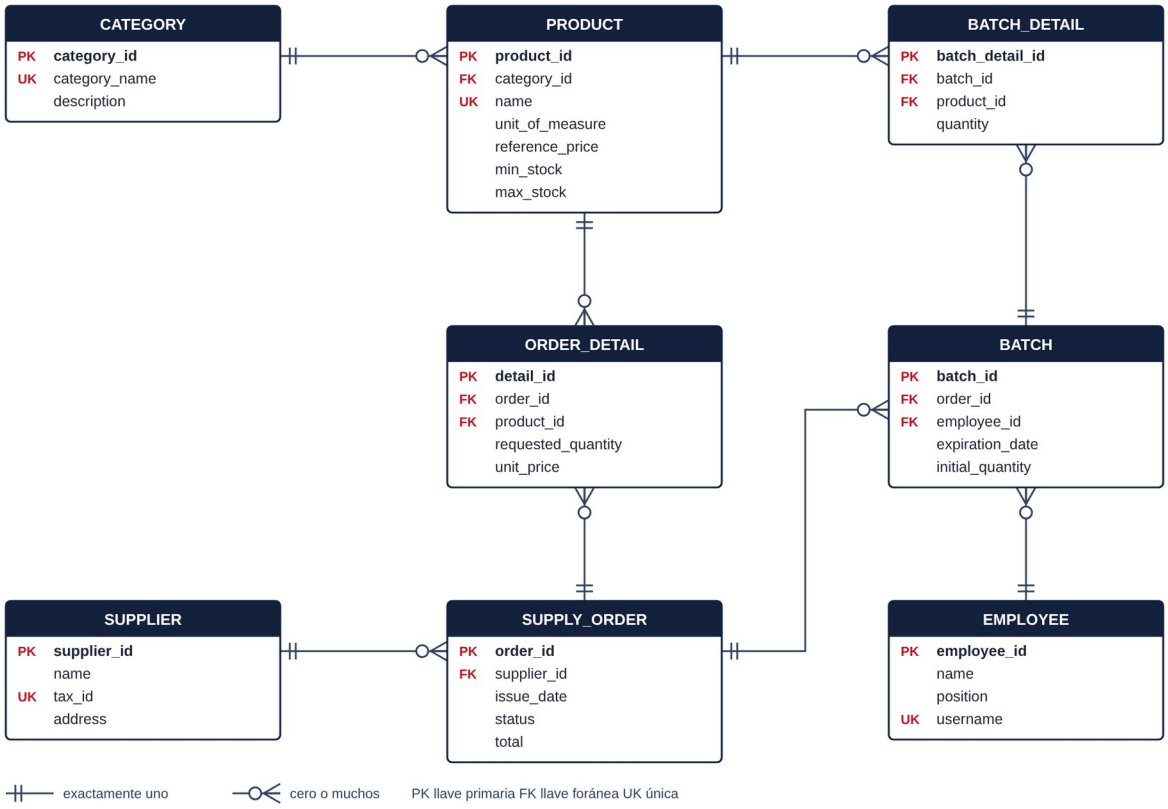
Refinamiento del modelo conceptual. La versión 1.1 incluía las entidades *Bodega* y *Movimiento de inventario*. Al socializar el alcance con el equipo se acotó el proyecto a un solo centro de distribución (S-01) y al proceso de entrada por abastecimiento (S-03). Con eso las entradas quedan representadas por los lotes y su detalle, y las entidades Bodega y Movimiento pasan a recomendaciones (sección 8).

2.6 Modelo lógico (Diagrama de ER)

El modelo lógico traduce las entidades a relaciones con llaves primarias (PK), llaves foráneas (FK) y llaves únicas (UK). Cada relación uno a muchos se implementa poniendo la PK del lado “uno” como FK en el lado “muchos”.

Figura 2

Modelo lógico con llaves primarias, foráneas y únicas



Nota. Elaboración propia.

Normalización

La normalización elimina la redundancia para que cada dato se guarde en un solo lugar (Codd, 1970; Elmasri y Navathe, 2016). Se verificó cada tabla en las tres primeras formas normales:

Tabla 5

Verificación de formas normales

Forma normal	Condición	Cómo se cumple en el modelo
1FN	Valores atómicos, sin grupos repetidos.	Los productos de una orden no se guardan como lista en SUPPLY_ORDER: cada uno es una fila de ORDER_DETAIL. Lo mismo ocurre con los productos de un lote en BATCH_DETAIL.
2FN	Cada atributo no clave depende de toda la llave.	Todas las tablas tienen PK simple (un solo identificador), por lo que no existen dependencias parciales. En ORDER_DETAIL, la combinación (order_id, product_id) es única y cantidad y precio dependen de la combinación completa.
3FN	Ningún atributo no clave depende de otro atributo no clave.	El nombre del proveedor no se repite en la orden: solo se guarda supplier_id. El nombre de la categoría no está en PRODUCT. El nombre del empleado no está en BATCH.
Excepción controlada	Dato derivado.	SUPPLY_ORDER.total se puede calcular desde ORDER_DETAIL. Se conserva por rendimiento de reportes y su valor lo mantiene el trigger T4, de modo que nunca queda inconsistente (RN-11).

Análisis CRUD

El análisis CRUD comprueba que cada proceso del negocio puede crear (C), leer (R), actualizar (U) o eliminar (D) los datos que necesita, y que ninguna tabla queda sin proceso que la alimente.

Tabla 6

Matriz CRUD por proceso

Proceso	CAT	PROD	SUPP	EMP	ORDER	O_DET	BATCH	B_DET	AUDIT
Gestionar catálogo	CRUD	CRUD				R		R	C
Gestionar proveedores			CRUD		R				C
Gestionar empleados				CRUD			R		C
Crear orden de abastecimiento		R	R		C	CRUD			C
Aprobar, recibir o cancelar orden					RU	R	R	R	C
Recibir lote		R		R	R	R	CRU	CRUD	
Consultar existencias y vencimientos	R	R	R	R	R	R	R	R	
Auditar cambios									R

Nota. AUDIT_LOG solo admite C (lo escriben los triggers) y R. El borrado (D) en tablas con hijos está limitado por ON DELETE RESTRICT.

Refinamiento del modelo lógico

El primer modelo físico, elaborado en Oracle SQL Developer Data Modeler, tenía las relaciones marcadas como identificadoras. Por eso las llaves aparecían como PF (PK y FK a la vez) y cada llave foránea apuntaba a la PK de la propia tabla. Se corrigieron diez errores antes de crear las tablas:

Tabla 7

Errores del modelo original y corrección aplicada

#	Tabla	Error	Corrección
1	Todas	Tipos de Oracle (VARCHAR2, NUMBER) que no existen en PostgreSQL.	VARCHAR y NUMERIC.
2	SUPPLY_ORDER	TIMESTAMP WITH LOCAL TIME ZONE (tipo de Oracle).	TIMESTAMPTZ.
3	PRODUCT	FK sobre product_id; no existía category_id.	Se agregó category_id como FK.
4	ORDER_DETAIL	detail_id era PK y FK a la vez; faltaban order_id y product_id.	FK propias y UNIQUE (order_id, product_id).
5	BATCH_DETAIL	Mismo error; un lote no podía tener varios productos.	FK batch_id y product_id con UNIQUE.
6	BATCH	FK al empleado duplicada y ambas sobre batch_id; faltaban employee_id y order_id.	Una sola FK por relación.
7	SUPPLY_ORDER	Nombre mal escrito (SUPLLY_ORDER) y FK sobre order_id.	Se renombró y se agregó supplier_id.
8	SUPPLY_ORDER	TOTAL NUMBER(10,2) no alcanza para órdenes de más de $99.999.999,99.	NUMERIC(12,2).
9	Todas	Sin UNIQUE ni CHECK: aceptaba precios negativos y NIT repetidos.	CHECK por dominio y UNIQUE de negocio.
10	Todas	Regla de borrado sin definir.	ON DELETE RESTRICT ON UPDATE RESTRICT.

3. Diseño

3.1 Especificaciones de diseño

Las especificaciones de diseño fijan las convenciones y decisiones que se aplicaron a todas las tablas:

Tabla 8

Decisiones de diseño

Decisión	Justificación
Esquema propio `inventario_postobon`	Separa el proyecto de otros esquemas de la base (como los talleres) y facilita respaldarlo.
Nombres en inglés y snake_case	Se conservan los nombres del modelo físico original para mantener la trazabilidad con el diagrama del equipo.
PK `INTEGER GENERATED ALWAYS AS IDENTITY`	El motor asigna el identificador; nadie puede inventarlo ni cambiarlo (PGDG, 2024b).
Restricciones con nombre (`tabla_columna_ck`, `_fk`, `_uk`)	El mensaje de error dice exactamente qué regla se violó, lo que facilita las pruebas y la sustentación.
`ON DELETE RESTRICT ON UPDATE RESTRICT` en todas las FK	Requisito del docente: ningún borrado en cascada. Un padre con hijos no se puede eliminar.
Reglas de varios pasos en triggers PL/pgSQL	Un CHECK solo ve una fila; reglas como “no recibir más de lo pedido” necesitan consultar otras tablas (PGDG, 2024c).
Índices sobre cada FK y columnas de filtro	Las validaciones de los triggers y los reportes no recorren tablas completas.
Tabla de auditoría con JSONB	Guarda la fila anterior y la nueva sin depender de la estructura de cada tabla.
Montos en NUMERIC y fechas en TIMESTAMPTZ	NUMERIC evita errores de redondeo en dinero; TIMESTAMPTZ guarda la zona horaria (America/Bogota).

Diccionario de datos. La sección 3.2 presenta, para cada tabla, sus columnas con tipo, tamaño, tipo de llave y opcionalidad, junto con cinco filas de ejemplo reales de la base cargada.

3.2 Transformación del modelo lógico a diagrama de tablas e instancias

Cada entidad del modelo lógico se transformó en una tabla. Los gráficos de instancia muestran datos de ejemplo del negocio para detectar errores de asignación antes de crear las tablas.

Convenciones: **PK** llave primaria, **FK** llave foránea, **UK** llave única; en opcionalidad, * significa obligatorio.

Tabla 9

CATEGORY — Categorías de producto (nombre abreviado: cat)

Tipo de clave	Opcionalidad	Columna	Tipo de dato	Tamaño
PK	*	category_id	INTEGER	
UK	*	category_name	VARCHAR	50
	*	description	VARCHAR	200

Nota. Restricciones CHECK: category_desc_ck; category_name_ck.

Tabla 10

Datos de ejemplo de CATEGORY

category_id	category_name	description
1	GASEOSAS DE SABORES	Bebidas carbonatadas de sabores a frutas: Postobón, Colom…
2	GASEOSAS COLA	Bebidas carbonatadas tipo cola: Pepsi, Pepsi Black, Fresk…
3	GASEOSAS LIMA LIMON	Bebidas carbonatadas lima-limón: 7Up, Bretaña
4	GASEOSAS CERO AZUCAR	Versiones sin azúcar de la línea de gaseosas
5	AGUA NATURAL	Agua potable tratada sin gas: Cristal, Oasis

Nota. Cinco primeras filas de la base cargada.

Tabla 11

PRODUCT — Productos (nombre abreviado: prod)

Tipo de clave	Opcionalidad	Columna	Tipo de dato	Tamaño
PK	*	product_id	INTEGER	
FK	*	category_id	INTEGER	
UK	*	name	VARCHAR	50
	*	unit_of_measure	VARCHAR	20
	*	reference_price	NUMERIC	10,2
	*	min_stock	INTEGER	
	*	max_stock	INTEGER	

Nota. Restricciones CHECK: product_max_ck; product_min_ck; product_name_ck; product_price_ck; product_unit_ck.

Tabla 12

Datos de ejemplo de PRODUCT

product_id	category_id	name	unit_of_measure	reference_price	min_stock	max_stock
1	3	7UP REGULAR 3 L PET	UNIDAD	9950.00	200	800
2	1	POPULAR NARANJA 500 ML PET X12	PACA	14660.00	200	1600
3	3	7UP REGULAR 1.5 L PET X12	PACA	58410.00	50	500
4	7	H2OH MANDARINA 3 L PET X12	PACA	98620.00	20	100

product_id	category_id	name	unit_of_measure	reference_price	min_stock	max_stock
5	11	SQUASH MORA 250 ML PET X12	PACA	13400.00	20	160

Nota. Cinco primeras filas de la base cargada.

Tabla 13

SUPPLIER — Proveedores (nombre abreviado: supp)

Tipo de clave	Opcionalidad	Columna	Tipo de dato	Tamaño
PK	*	supplier_id	INTEGER	
	*	name	VARCHAR	50
UK	*	tax_id	VARCHAR	20
	*	address	VARCHAR	50

Nota. Restricciones CHECK: supplier_address_ck; supplier_name_ck; supplier_tax_id_ck.

Tabla 14

Datos de ejemplo de SUPPLIER

supplier_id	name	tax_id	address
1	INVERSIONES LA PLAYA S.A.S.	846721674-7	KM 15 VÍA OCAÑA LA PLAYA
2	INVERSIONES LA ESPERANZA & CIA S.A.S.	825604412-7	KM 19 VÍA OCAÑA CONVENCIÓN
3	INDUSTRIAS LA PLAYA Y CIA LTDA	889707012-1	KM 12 VÍA OCAÑA CONVENCIÓN
4	CARTONES LOS PATIOS LTDA	824488171-5	DG 70 # 66-8 AGUACHICA
5	AGROINDUSTRIAL EL CARMEN S.A.	806864693-5	AV 94 # 93-73 CÚCUTA

Nota. Cinco primeras filas de la base cargada.

Tabla 15

EMPLOYEE — Empleados (nombre abreviado: emp)

Tipo de clave	Opcionalidad	Columna	Tipo de dato	Tamaño
PK	*	employee_id	INTEGER	
	*	name	VARCHAR	50
	*	position	VARCHAR	50
UK	*	username	VARCHAR	50

Nota. Restricciones CHECK: employee_name_ck; employee_position_ck; employee_username_ck.

Tabla 16

Datos de ejemplo de EMPLOYEE

employee_id	name	position	username
1	HERNÁN ANTONIO LEÓN ANGARITA	AUXILIAR DE BODEGA	haleona
2	DAIRO ALBERTO ORTEGA ARÉVALO	AUXILIAR DE BODEGA	daortegaa

employee_id	name	position	username
3	FABIÁN ANDRÉS MARTÍNEZ REYES	AUXILIAR DE BODEGA	famartinezr
4	DAIRO JOSÉ CARRASCAL QUINTERO	ANALISTA DE INVENTARIO	djcarrascalq
5	EDWIN VELÁSQUEZ PARADA	ANALISTA DE INVENTARIO	evelasquezp

Nota. Cinco primeras filas de la base cargada.

Tabla 17

SUPPLY_ORDER — Órdenes de abastecimiento (nombre abreviado: ord)

Tipo de clave	Opcionalidad	Columna	Tipo de dato	Tamaño
PK	*	order_id	INTEGER	
FK	*	supplier_id	INTEGER	
	*	issue_date	TIMESTAMPTZ	
	*	status	VARCHAR	20
	*	total	NUMERIC	12,2

Nota. Restricciones CHECK: supply_order_date_ck; supply_order_status_ck; supply_order_total_ck.

Tabla 18

Datos de ejemplo de SUPPLY_ORDER

order_id	supplier_id	issue_date	status	total
1	13	2023-01-04 00:06	CANCELADA	856449.00
2	4	2023-01-06 01:21	CANCELADA	10817532.00
4	43	2023-01-08 02:21	RECIBIDA	1032468.00
479	10	2026-06-27 09:19	APROBADA	422060.00
651	5	2026-09-10 02:00	PENDIENTE	6933112.80

Nota. Cinco primeras filas de la base cargada. En SUPPLY_ORDER se muestra una orden de cada estado.

Tabla 19

ORDER_DETAIL — Detalle de las órdenes (nombre abreviado: odet)

Tipo de clave	Opcionalidad	Columna	Tipo de dato	Tamaño
PK	*	detail_id	INTEGER	
FK, UK	*	order_id	INTEGER	
FK, UK	*	product_id	INTEGER	
	*	requested_quantity	INTEGER	
	*	unit_price	NUMERIC	10,2

Nota. Restricciones CHECK: order_detail_price_ck; order_detail_qty_ck.

Tabla 20

Datos de ejemplo de ORDER_DETAIL

detail_id	order_id	product_id	requested_quantity	unit_price
1	1	69	160	1470.24
2	1	28	20	31060.53
3	2	3	200	54087.66
4	3	15	72	2277.02
5	4	85	50	20649.36

Nota. Cinco primeras filas de la base cargada.

Tabla 21

BATCH — Lotes recibidos (nombre abreviado: bat)

Tipo de clave	Opcionalidad	Columna	Tipo de dato	Tamaño
PK	*	batch_id	INTEGER	
FK	*	order_id	INTEGER	
FK	*	employee_id	INTEGER	
	*	expiration_date	DATE	
	*	initial_quantity	INTEGER	

Nota. Restricciones CHECK: batch_qty_ck.

Tabla 22

Datos de ejemplo de BATCH

batch_id	order_id	employee_id	expiration_date	initial_quantity
1	4	50	2023-12-20	50
2	7	7	2023-05-14	127
3	7	23	2023-10-24	41
4	9	4	2023-05-14	10
5	12	26	2023-09-21	70

Nota. Cinco primeras filas de la base cargada.

Tabla 23

BATCH_DETAIL — Detalle de los lotes (nombre abreviado: bdet)

Tipo de clave	Opcionalidad	Columna	Tipo de dato	Tamaño
PK	*	batch_detail_id	INTEGER	
FK, UK	*	batch_id	INTEGER	
FK, UK	*	product_id	INTEGER	
	*	quantity	INTEGER	

Nota. Restricciones CHECK: batch_detail_qty_ck.

Tabla 24

Datos de ejemplo de BATCH_DETAIL

batch_detail_id	batch_id	product_id	quantity
1	1	85	50
2	2	76	127
3	3	33	41
4	4	46	10
5	5	49	50

Nota. Cinco primeras filas de la base cargada.

Tabla 25

AUDIT_LOG — Auditoría de cambios (nombre abreviado: aud)

Tipo de clave	Opcionalidad	Columna	Tipo de dato	Tamaño
PK	*	audit_id	BIGINT	
	*	table_name	VARCHAR	30
	*	operation	CHAR	1
	*	record_id	INTEGER	
	o	old_data	JSONB	
	o	new_data	JSONB	
	*	changed_at	TIMESTAMPTZ	
	*	db_user	VARCHAR	50

Nota. Restricciones CHECK: audit_log_operation_ck.

Tabla 26

Datos de ejemplo de AUDIT_LOG

audit_id	table_name	operation	record_id	new_data (extracto)	changed_at	db_user
1	category	I	1	{"category_id": 1, "description": "Bebid	2026-11-05 14:58	postgres
2	category	I	2	{"category_id": 2, "description": "Bebid	2026-11-05 14:58	postgres
3	category	I	3	{"category_id": 3, "description": "Bebid	2026-11-05 14:58	postgres
4	category	I	4	{"category_id": 4, "description": "Versi	2026-11-05 14:58	postgres
5	category	I	5	{"category_id": 5, "description": "Agua	2026-11-05 14:58	postgres

Nota. Cinco primeras filas de la base cargada.

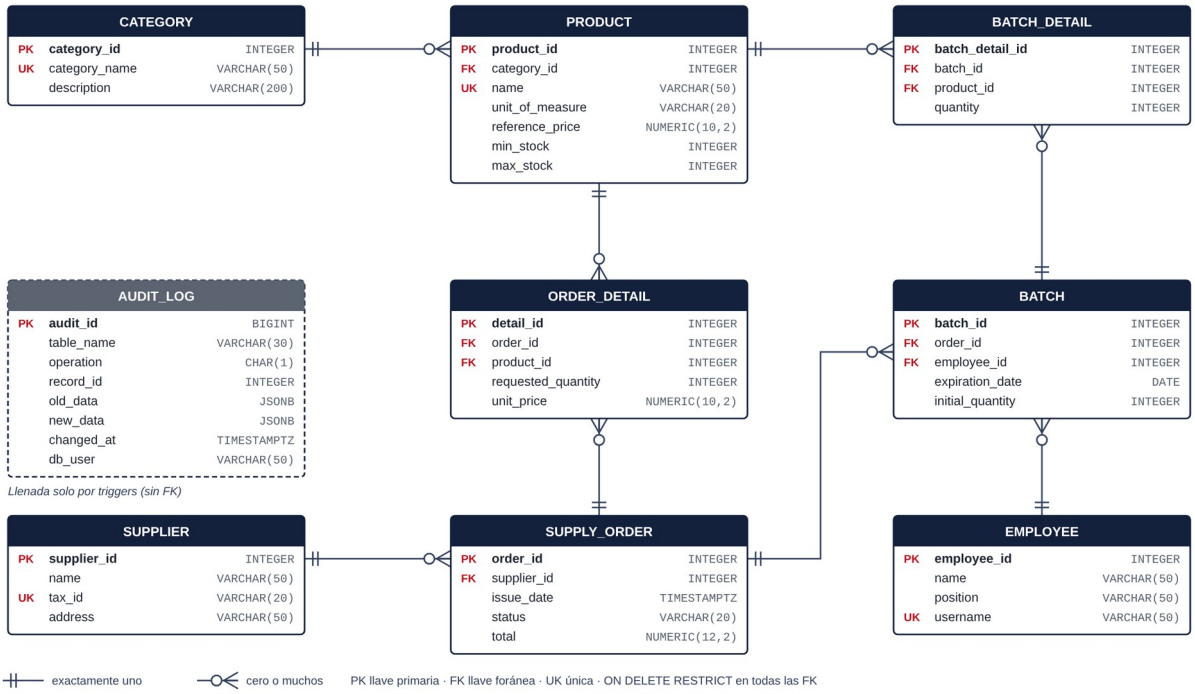
4. Creación

4.1 Modelo físico

El modelo físico define las tablas en PostgreSQL 16 con sus tipos de datos definitivos, restricciones e índices. A diferencia del modelo lógico incluye la tabla técnica AUDIT_LOG, que no tiene llaves foráneas porque registra cambios de varias tablas a la vez.

Figura 3

Modelo físico implementado en PostgreSQL 16



Nota. Elaboración propia. Todas las llaves foráneas usan ON DELETE RESTRICT ON UPDATE RESTRICT.

Refinamiento del modelo físico. Respecto al modelo lógico se agregaron: índices sobre todas las FK y sobre estado, fecha y vencimiento; índices únicos sobre `upper(name)` para que “Gaseosas” y “GASEOSAS” cuenten como el mismo nombre; valores por defecto (`status = PENDIENTE`, `issue_date = now()`, `total = 0`); y comentarios en cada tabla, visibles en pgAdmin. La base final tiene 25 índices, 9 funciones y 29 triggers.

4.2 Script DDL y DML

Los scripts se ejecutan en orden en el Query Tool de pgAdmin. Todos se probaron sobre una base vacía y corren sin errores.

Tabla 27

Scripts del proyecto

Archivo	Tipo	Contenido
01_ddl.sql	DDL	Esquema, 9 tablas, PK, FK con RESTRICT, CHECK, UNIQUE, índices y comentarios.
02_triggers.sql	DDL	9 funciones PL/pgSQL y 29 triggers (reglas de negocio, total automático, auditoría y bloqueo de TRUNCATE).
03a_datos_base_ejemplo.sql	DML	Datos base: 20 categorías, 50 proveedores y 50 empleados.
03_datos.sql	DML	100 productos, 1.000 órdenes, 1.500 detalles, 500 lotes y 500 detalles de lote, siguiendo el ciclo de vida real de cada orden.
03b_importar_csv.sql	DML	Alternativa: importación con COPY desde los CSV del Excel definitivo (Anexo 1, E7).
04_pruebas_ataque.sql	Pruebas	62 intentos de romper la base y 5 verificaciones; deshace todo al terminar.
05_consultas.sql	DQL	3 vistas de reporte y 12 consultas de verificación.
07_seguridad.sql	DCL	Roles `consulta_inventario` y `app_inventario` con permisos mínimos.
08_validacion_datos.sql	DQL	Validación de integridad de los datos importados (Anexo 1, E8).

El fragmento siguiente muestra la creación de una tabla con todas sus restricciones. Las demás siguen el mismo patrón:

Tabla 28

Fragmento de 01_ddl.sql: tabla PRODUCT

```
CREATE TABLE product (
    product_id       INTEGER GENERATED ALWAYS AS IDENTITY,
    category_id      INTEGER       NOT NULL,
    name             VARCHAR(50)   NOT NULL,
    unit_of_measure  VARCHAR(20)   NOT NULL,
    reference_price  NUMERIC(10,2) NOT NULL,
    min_stock        INTEGER       NOT NULL,
    max_stock        INTEGER       NOT NULL,
    CONSTRAINT product_pk PRIMARY KEY (product_id),
    CONSTRAINT product_category_fk FOREIGN KEY (category_id)
        REFERENCES category (category_id) ON DELETE RESTRICT ON UPDATE RESTRICT,
    CONSTRAINT product_name_ck  CHECK (length(btrim(name)) >= 3),
    CONSTRAINT product_unit_ck  CHECK (unit_of_measure IN
        ('UNIDAD','PACA','CANASTA','CAJA','SIXPACK','FARDO','BOLSA')),
    CONSTRAINT product_price_ck CHECK (reference_price > 0),
    CONSTRAINT product_min_ck   CHECK (min_stock >= 0),
    CONSTRAINT product_max_ck   CHECK (max_stock > min_stock)
```

```
);
CREATE UNIQUE INDEX product_name_uk ON product (upper(name));
```

Triggers

Los triggers siguen la estructura vista en clase: una función `RETURNS trigger` y un `CREATE TRIGGER` que la asocia a la tabla (Cuesta Quintero, 2026; PGDG, 2024c).

Tabla 29

Funciones disparadoras y triggers

#	Función	Momento y nivel	Regla que implementa
T1	fn_normalizar_texto	BEFORE INSERT/UPDATE, fila	RN-04: quita espacios y unifica mayúsculas.
T2	fn_supply_order_reglas	BEFORE INSERT/UPDATE, fila	RN-07, RN-08, RN-09, RN-11: ciclo de vida, fecha y total.
T3	fn_order_detail_reglas	BEFORE INSERT/UPDATE/DELETE, fila	RN-10: detalle editable solo en PENDIENTE; bloquea la orden con FOR UPDATE.
T4	fn_order_detail_total	AFTER INSERT/UPDATE/DELETE, fila	RN-11: recalcula el total de la orden.
T5	fn_batch_reglas	BEFORE INSERT/UPDATE/DELETE, fila	RN-12: lotes solo de órdenes APROBADAS; vencimiento válido.
T6	fn_batch_detail_reglas	BEFORE INSERT/UPDATE/DELETE, fila	RN-14: no recibir lo no pedido ni de más.
T7	fn_auditar	AFTER INSERT/UPDATE/DELETE, fila, con WHEN	RN-16: guarda OLD y NEW en JSONB.
T8	fn_auditoria_inalterable	BEFORE UPDATE/DELETE/TRUNCATE	RN-16: la auditoría es de solo lectura.
T9	fn_bloquear_truncate	BEFORE TRUNCATE, sentencia	RN-15: impide vaciar tablas con TRUNCATE … CASCADE.

Tabla 30

Fragmento de 02_triggers.sql: total automático de la orden (T4)

```
CREATE OR REPLACE FUNCTION fn_order_detail_total() RETURNS trigger AS $$
BEGIN
    UPDATE supply_order SET total = 0     -- T2 reemplaza el 0 por la suma real
     WHERE order_id = CASE WHEN TG_OP = 'DELETE' THEN OLD.order_id ELSE NEW.order_id END;
    RETURN NULL;                          -- AFTER: el valor de retorno se ignora
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_order_detail_total
    AFTER INSERT OR UPDATE OR DELETE ON order_detail
    FOR EACH ROW EXECUTE FUNCTION fn_order_detail_total();
```

Carga de datos (DML)

La carga se hace con los triggers activos y dentro de una sola transacción: si una fila viola una regla, no queda nada a medias. Las órdenes se insertan PENDIENTES y luego recorren su ciclo real

(detalle, aprobación, lotes, recepción). Las tablas CATEGORY, SUPPLIER y EMPLOYEE las llena el equipo; el script toma sus identificadores por posición, de modo que funciona con cualquier dato base. El proceso de generación con IA y su validación se documentan en el Anexo 1.

Tabla 31

Volumen de datos cargado

Tabla	Filas	Origen
category	20	Datos base del equipo (03a)
supplier	50	Datos base del equipo (03a)
employee	50	Datos base del equipo (03a)
product	100	Generado con IA y validado (03)
supply_order	1.000	Generado con IA y validado (03)
order_detail	1.500	Generado con IA y validado (03)
batch	500	Generado con IA y validado (03)
batch_detail	500	Generado con IA y validado (03)
audit_log	2.392	Generada por los triggers durante la carga

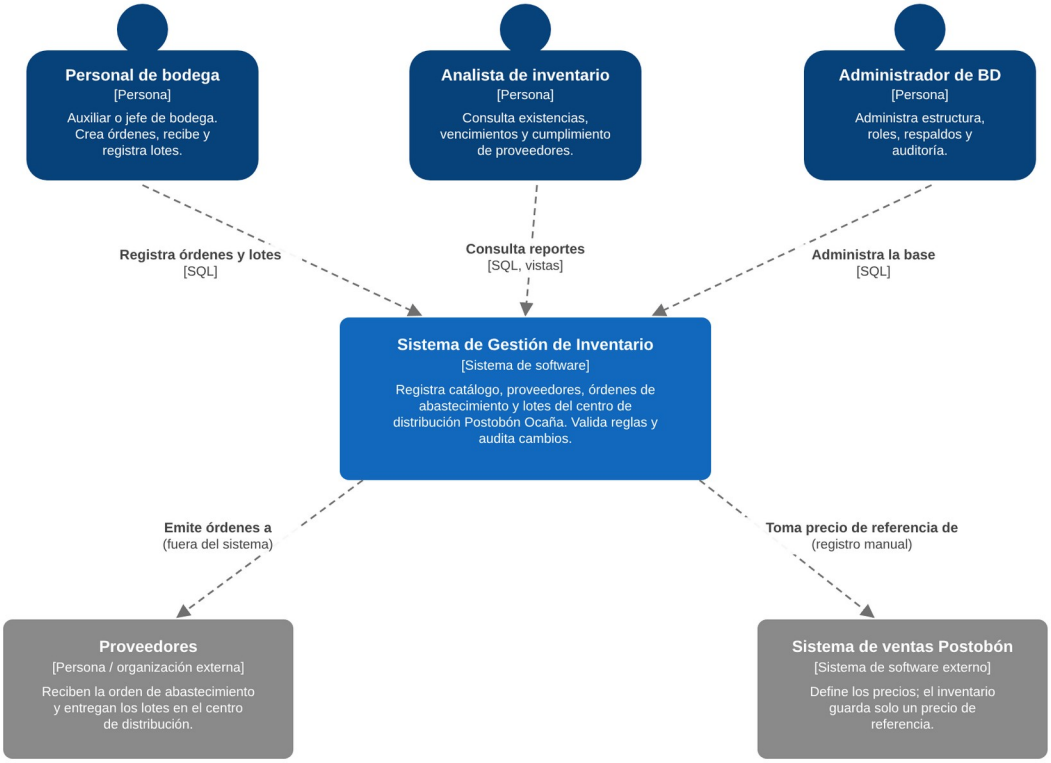
4.3 Modelo C4

El modelo C4 describe la arquitectura del software en niveles de detalle progresivo (Brown, s. f.). El nivel 1 muestra el sistema en su entorno; el nivel 2 muestra los contenedores que lo componen.

Nivel 1: Diagrama de contexto del sistema

Figura 4

Diagrama de contexto del sistema (C4, nivel 1)

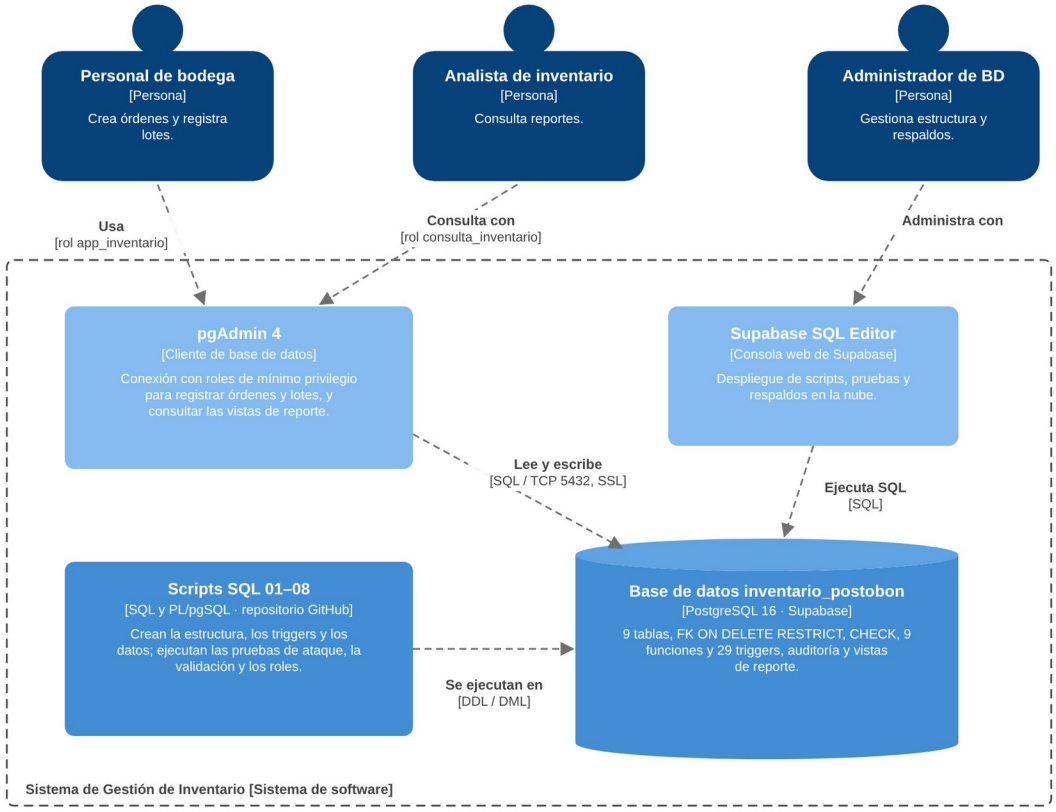


Nota. Elaboración propia con la notación de Brown (s. f.). Los sistemas en gris son externos y no se integran automáticamente (supuestos S-02 y S-03).

Nivel 2: Diagrama de contenedores

Figura 5

Diagrama de contenedores (C4, nivel 2)



Nota. Elaboración propia. Los usuarios se conectan a la base con roles de permisos mínimos (07_seguridad.sql); las reglas de negocio se aplican en la base, sin importar el cliente que se use.

4.4 Repositorio GitHub

URL del repositorio: https://github.com/julian-parada/inventario-postobon

El repositorio contiene todos los archivos necesarios para reproducir el proyecto, organizados así:

Tabla 32

Estructura del repositorio

```
inventario-postobon/
├── README.md                    instrucciones de instalación y orden de ejecución
├── docs/
│   ├── Documento_especificacion_BD.docx
│   ├── diagramas/               conceptual, lógico, físico, C4 (PNG y SVG)
```

```
│   └── anexo1/                E3_excel_inicial_IA.xlsx, E6_excel_definitivo.xlsx
├── sql/
│   ├── 01_ddl.sql   02_triggers.sql   03a_datos_base_ejemplo.sql   03_datos.sql
│   ├── 03b_importar_csv.sql   04_pruebas_ataque.sql   05_consultas.sql
│   ├── 07_seguridad.sql   08_validacion_datos.sql
│   └── backup_inventario_postobon.backup
├── datos/
│   ├── csv/                   8 archivos CSV para COPY
│   └── generador/generar_datos.py
```

5. Transición

5.1 Pruebas de aceptación de usuario (UAT)

Las pruebas de aceptación de usuario validan que la base de datos cumple los requisitos antes de su despliegue en producción. Todas las pruebas se ejecutaron sobre una base limpia, cargada con los scripts 01 a 03, y los resultados obtenidos son los que devolvió PostgreSQL.

Tabla 33

Informe UAT

Campo	Valor
Fecha del informe	05/11/2026
Responsable	Equipo del proyecto (Parada Reyes, Velásquez Gómez)
Versión del sistema	Esquema inventario_postobon v2.0 (scripts 01–08)
Entorno de prueba	Servidor de desarrollo: PostgreSQL 16, aislamiento READ COMMITTED, zona America/Bogota

Estrategia de pruebas

Cada caso se ejecuta dentro de una transacción que se deshace al final (ROLLBACK), para que una prueba no afecte a la siguiente. Una prueba de integridad se aprueba cuando la base **rechaza** la operación inválida con la restricción esperada.

Tabla 34

Tipos de pruebas

Tipo de prueba	Descripción
Integridad de datos	Verifica que inserciones, actualizaciones y eliminaciones cumplen las restricciones y reglas del negocio.
Consultas y reportes	Valida que las consultas y vistas devuelven los resultados esperados.
Concurrencia	Evalúa el comportamiento con varios usuarios modificando los mismos datos.
Seguridad	Verifica autorización por roles e inmunidad a inyección SQL.
Rendimiento	Mide tiempos de ejecución con un millón de registros y el efecto de los índices.
Recuperación ante fallos	Simula fallos y verifica la recuperación de los datos.

Pruebas de integridad de datos

Tabla 35

Casos de prueba de integridad

ID	Caso de prueba	Consulta SQL	Resultado esperado	Resultado obtenido	Estado
INT-01	Clave primaria manual	INSERT INTO category (category_id, …) VALUES (1, …)	Error: el ID lo asigna la base	Rechazado (428C9)	Aprobado
INT-02	FK inexistente	INSERT INTO product (category_id=99999, …)	Error de llave foránea	Rechazado (23503, product_category_fk)	Aprobado
INT-03	NIT duplicado	INSERT INTO supplier con tax_id existente	Error de unicidad	Rechazado (23505, supplier_tax_id_uk)	Aprobado
INT-04	Precio negativo	INSERT INTO product (reference_price = -2500)	Error CHECK de precio	Rechazado (23514, product_price_ck)	Aprobado
INT-05	Stock máximo < mínimo	INSERT … min_stock 500, max_stock 100	Error CHECK de stock	Rechazado (23514, product_max_ck)	Aprobado
INT-06	NIT con formato inválido	INSERT INTO supplier (tax_id = '123')	Error CHECK de NIT	Rechazado (23514, supplier_tax_id_ck)	Aprobado
INT-07	Eliminar categoría con productos	DELETE FROM category WHERE category_id = 1	Error RESTRICT, sin cascada	Rechazado (23503, product_category_fk)	Aprobado
INT-08	Vaciar con cascada	TRUNCATE category CASCADE	Bloqueado por trigger T9	Rechazado (42501)	Aprobado
INT-09	Retroceder estado	UPDATE supply_order SET status='PENDIENTE' (orden RECIBIDA)	Bloqueado por trigger T2	Rechazado (23514)	Aprobado
INT-10	Lote de orden PENDIENTE	INSERT INTO batch (orden PENDIENTE)	Bloqueado por trigger T5	Rechazado (23514)	Aprobado
INT-11	Recibir más de lo pedido	INSERT INTO batch_detail (cantidad > pedida)	Bloqueado por trigger T6	Rechazado (23514)	Aprobado
INT-12	Borrar auditoría	DELETE FROM audit_log	Bloqueado por trigger T8	Rechazado (42501)	Aprobado
INT-13	Cargo inexistente	INSERT INTO employee (position='GERENTE')	Error CHECK de cargo	Rechazado (23514, employee_position_ck)	Aprobado
INT-14	Falsificar el total	UPDATE supply_order SET total = 1	El total se recalcula	Orden 4: total queda 1032468.00 = suma del detalle 1032468.00	Aprobado

Observaciones. 14 de 14 casos aprobados. Además, el script `04_pruebas_ataque.sql` ejecuta 62 ataques (valores nulos, duplicados disfrazados con mayúsculas, desbordamiento de tipos, cambios de llave, ciclos de estado inválidos, borrados masivos) y 5 verificaciones de reglas automáticas: los 62 fueron bloqueados y las 5 verificaciones dieron OK. No se identificaron problemas críticos.

Pruebas de consultas y reportes

Tabla 36

Casos de prueba de consultas

ID	Caso de prueba	Consulta SQL	Resultado esperado	Resultado obtenido	Estado
QRY-01	Totales de orden coherentes (RI-04)	Órdenes cuyo total ≠ suma del detalle	0 filas	0 filas	Aprobado
QRY-02	Nada recibido de más (RN-14)	Recibido > pedido por producto y orden	0 filas	0 filas	Aprobado
QRY-03	Estado de existencias (RI-02)	SELECT estado, count(*) FROM v_existencias GROUP BY estado	Productos clasificados por estado	NORMAL: 45; BAJO EL MINIMO: 33; SIN EXISTENCIA: 14; SOBRE EL MAXIMO: 8	Aprobado
QRY-04	Lotes por vencer en 60 días (RI-08)	SELECT * FROM v_lotes_por_vencer	Lista de lotes próximos a vencer	54 filas	Aprobado
QRY-05	Compras por proveedor (RI-05)	SUM(total) GROUP BY proveedor (RECIBIDA)	Un valor por proveedor	50 proveedores	Aprobado
QRY-06	Proveedor de cada lote (RI-07)	batch JOIN supply_order JOIN supplier	Un proveedor por lote	500 lotes	Aprobado
QRY-07	Órdenes por estado (RI-11)	SELECT status, count(*), sum(total) GROUP BY status	4 estados	4 estados	Aprobado
QRY-08	Ninguna FK en cascada (RN-15)	pg_constraint con confdeltype ≠ RESTRICT	0 filas	0 filas	Aprobado
QRY-09	Cumplimiento de proveedores (RI-10)	SELECT * FROM v_cumplimiento_proveedor	% recibido por proveedor	10 filas (mínimo 68.35 %)	Aprobado
QRY-10	Lotes por empleado (RI-09)	batch JOIN employee GROUP BY nombre	Un conteo por empleado	50 empleados	Aprobado

Observaciones. 10 de 10 casos aprobados. Cada requisito de información (RI-01 a RI-11) tiene al menos una consulta que lo responde. Todas las consultas se ejecutaron en menos de 10 ms.

Pruebas de concurrencia

Tabla 37

Casos de prueba de concurrencia

ID	Caso de prueba	Acción	Resultado esperado	Resultado obtenido	Estado
CON-01	Dos usuarios actualizan el mismo producto	Sesión A: UPDATE sin confirmar; sesión B: UPDATE del mismo ID con lock_timeout = 2 s	B espera o recibe error de bloqueo	B esperó 2.0 s y recibió 55P03 (lock timeout)	Aprobado
CON-02	Lectura mientras otra sesión escribe	A: UPDATE precio = 99999 sin confirmar; B: SELECT del mismo producto	B ve el dato confirmado (READ COMMITTED)	B leyó 9950.00 (valor confirmado) en 0.2 ms, sin bloquearse	Aprobado
CON-03	Dos usuarios agregan productos a la misma orden	A y B: INSERT en order_detail de la misma orden	El trigger T3 serializa con FOR UPDATE	B esperó el bloqueo de A (55P03); total final correcto: 10000.00	Aprobado

Observaciones. 3 de 3 casos aprobados. El bloqueo de fila evita que dos usuarios pisen el mismo dato y que el total de una orden quede mal calculado. La sesión que espera recibe el error de bloqueo y puede reintentar la operación.

Pruebas de seguridad

Tabla 38

Casos de prueba de seguridad

ID	Caso de prueba	Acción	Resultado esperado	Resultado obtenido	Estado
SEC-01	Usuario sin permisos accede a datos sensibles	Rol consulta_inventario: SELECT * FROM audit_log	Acceso denegado	Rechazado (42501: permiso denegado)	Aprobado
SEC-02	Usuario de consulta intenta borrar	Rol consulta_inventario: DELETE FROM product	Acceso denegado	Rechazado (42501: permiso denegado)	Aprobado
SEC-03	Usuario sin privilegio altera la estructura	ALTER TABLE product DISABLE TRIGGER ALL; DROP TABLE category	Acceso denegado	Rechazado (42501: debe ser dueño de la tabla)	Aprobado
SEC-04	SQL Injection	Buscar ' OR 1=1 -- en un campo de texto (consulta parametrizada)	La entrada se trata como texto	Devuelve 0 de 100 productos	Aprobado

Observaciones. 4 de 4 casos aprobados. Hallazgo crítico documentado: el superusuario `postgres` puede desactivar triggers o borrar tablas; por eso el personal de bodega y los analistas deben conectarse con los roles de `07_seguridad.sql` y nunca con el superusuario.

Pruebas de rendimiento

Tabla 39

Casos de prueba de rendimiento

ID	Caso de prueba	Acción	Resultado esperado	Resultado obtenido	Estado
PER-01	Consultas con más de 1 millón de registros	Tabla de prueba de 1.000.000 de detalles; SUM y COUNT agrupados por producto	Respuesta < 2 s	Carga en 2.4 s; consulta en 293.2 ms	Aprobado
PER-02	Indexación de consultas	Buscar una orden con y sin índice (EXPLAIN ANALYZE)	Mejora significativa	Sin índice: 47.8 ms (Seq Scan); con índice: 0.2 ms (Bitmap Index Scan)	Aprobado

Observaciones. 2 de 2 casos aprobados. El índice redujo el tiempo de búsqueda unas 239 veces. La tabla de un millón de filas se creó en un esquema temporal y se eliminó después de la prueba.

Pruebas de recuperación ante fallos

Tabla 40

Casos de prueba de recuperación

ID	Caso de prueba	Acción	Resultado esperado	Resultado obtenido	Estado
REC-01	Restauración de backup	pg_dump (formato custom) y pg_restore en una base nueva	Datos sin pérdida	Mismas filas en las 5 tablas, mismo valor total ($3.119.625.889,20) y 29 triggers restaurados	Aprobado
REC-02	Caída del servidor en transacción activa	Insertar 200 órdenes sin confirmar y apagar el servidor en modo immediate	Consistencia tras la recuperación	Recuperación automática por WAL; 1.000 órdenes antes y después; 0 totales descuadrados	Aprobado

Observaciones. 2 de 2 casos aprobados. La transacción interrumpida no dejó ninguna fila a medias: PostgreSQL garantiza atomicidad y durabilidad mediante el registro de escritura anticipada (PGDG, 2024d).

Criterios de éxito y aprobación

- El 100 % de los casos de prueba críticos pasaron: 35 de 35 casos UAT y 62 de 62 ataques del script 04.
- No se detectaron errores críticos en integridad, seguridad ni concurrencia.
- El rendimiento es aceptable: ninguna consulta del sistema supera 10 ms y la prueba de un millón de registros respondió en menos de 0,5 s.
- Recomendación antes del despliegue: crear los roles de 07_seguridad.sql en Supabase y que cada usuario se conecte con el rol que le corresponde.

5.2 Matriz de trazabilidad

La matriz relaciona cada requisito con las reglas, tablas, objetos de base de datos y pruebas que lo cubren, para comprobar la coherencia entre análisis, diseño, implementación y pruebas.

Tabla 41

Matriz de trazabilidad

Requisito	Reglas	Tablas	Objeto / script	Pruebas
RI-01	RN-01 a RN-04	category, product	01_ddl (CHECK, UK), T1	INT-02, INT-04, INT-05, INT-07
RI-02	RN-02, RN-14	product, batch, batch_detail	Vista v_existencias (05)	QRY-03
RI-03	RN-05	supplier	01_ddl (UK, CHECK)	INT-03, INT-06
RI-04	RN-07 a RN-11	supply_order, order_detail	T2, T3, T4	INT-09, INT-14, QRY-01,

Requisito	Reglas	Tablas	Objeto / script	Pruebas
				CON-03
RI-05	RN-07	supplier, supply_order	Consulta C13 (05)	QRY-05
RI-06	RN-12 a RN-14	batch, batch_detail, employee	T5, T6	INT-10, INT-11, QRY-02
RI-07	RN-07, RN-12	batch, supply_order, supplier	Consulta C14 (05)	QRY-06
RI-08	RN-12	batch, batch_detail, product	Vista v_lotes_por_vencer	QRY-04
RI-09	RN-13, RN-16	batch, employee, audit_log	T7, T8	INT-12, QRY-10, SEC-01
RI-10	RN-14	order_detail, batch_detail	Vista v_cumplimiento_proveedor	QRY-09
RI-11	RN-08, RN-11	supply_order	Consulta C2 (05)	QRY-07, PER-01
Transversal	RN-15	Todas	FK RESTRICT, T9	INT-07, INT-08, QRY-08, REC-01

6. Producción

6.1 Desplegar el modelo físico en la nube (Supabase)

Supabase ofrece una base PostgreSQL administrada en la nube, compatible con los scripts del proyecto (Supabase, s. f.). El despliegue se realiza con el siguiente procedimiento:

1. Crear una cuenta en https://supabase.com y un proyecto nuevo llamado `inventario-postobon`, con una contraseña segura para la base. El proyecto del equipo quedó en la región us-east-1.
2. Abrir el **SQL Editor** del proyecto y ejecutar, en orden, `01_ddl.sql`, `02_triggers.sql`, `03a_datos_base_ejemplo.sql`, `03_datos.sql`, `05_consultas.sql` y `07_seguridad.sql`.
3. En **Table Editor** verificar que aparecen las 9 tablas del esquema `inventario_postobon`.
4. Ejecutar `08_validacion_datos.sql` y comprobar los mismos resultados de la validación local (Anexo 1, E8).
5. Con el botón **Connect** del proyecto copiar la cadena de conexión del *Session pooler* (host, puerto, usuario y base), que se usa para conectarse desde pgAdmin. La conexión directa (*Direct connection*) solo funciona por IPv6, por eso se usa el pooler.
6. Ejecutar `04_pruebas_ataque.sql` en el SQL Editor y verificar 62 BLOQUEADO y 5 OK.

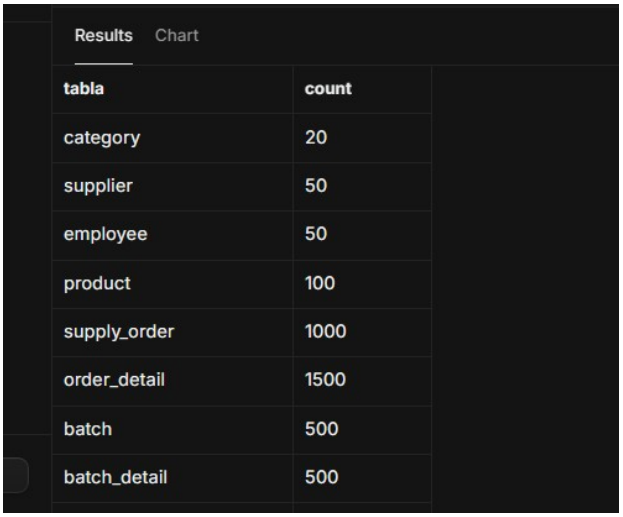
Tabla 42

Verificación del despliegue en Supabase

Verificación	Resultado esperado	Cómo se comprueba
Tablas creadas	9 tablas en inventario_postobon	Table Editor, seleccionando el esquema inventario_postobon
Datos cargados	20 / 50 / 50 / 100 / 1.000 / 1.500 / 500 / 500 filas	Conteo de filas por tabla en el SQL Editor (Figura 6)
Triggers activos	29 triggers	SELECT count(*) FROM pg_trigger … WHERE nspname = 'inventario_postobon' AND NOT tgisinternal
Pruebas de ataque	62 BLOQUEADO, 5 OK	Resumen final de 04_pruebas_ataque.sql

Figura 6

Conteo de filas por tabla en la base desplegada en Supabase



Nota. Captura del SQL Editor de Supabase. Coincide con el volumen cargado en la base local (sección 4.2).

Problemas encontrados en el despliegue y su solución

Los scripts se habían probado en pgAdmin, pero el SQL Editor de Supabase se comporta distinto en dos puntos. Ambos se corrigieron en los scripts del repositorio:

Tabla 43

Errores del despliegue en Supabase

Error	Causa	Solución
relation "map_category" does not exist (42P01) al ejecutar 03_datos.sql	El script usaba tablas temporales con ON COMMIT DROP; el SQL Editor maneja la transacción por su cuenta y las tablas desaparecían antes de usarse.	Las tablas puente pasaron a ser tablas normales que el mismo script borra al final. El conteo final quedó completo (Figura 6).
relation "audit_log" does not exist (42P01) al insertar un producto desde el SQL Editor	Las funciones de los triggers nombraban las tablas sin esquema y dependían del search_path de la sesión; en Supabase la sesión empieza en el esquema public.	Cada función se fijó con SET search_path = inventario_postobon, pg_temp. Ahora los triggers funcionan desde cualquier cliente.

6.2 Colocar en funcionamiento el sistema

La base desplegada en Supabase se opera desde pgAdmin 4, igual que la base local, registrando un servidor nuevo con los datos del *Session pooler* y la conexión cifrada (SSL) activada.

Tabla 44

Registro del servidor de Supabase en pgAdmin 4

Campo de pgAdmin	Valor
Host name/address	aws-0-<región>.pooler.supabase.com
Port	5432 (Session pooler)
Maintenance database	postgres
Username	postgres.<id-del-proyecto> (o el rol de 07_seguridad.sql)
SSL mode	require

Operaciones de verificación en funcionamiento, ejecutadas con `SET search_path TO inventario_postobon;`: consultar las vistas de reporte, crear un producto válido, intentar un producto con precio negativo (debe rechazarse), crear y aprobar una orden, y revisar que la operación quedó en la auditoría.

7. Conclusión

La base de datos cumple las necesidades del centro de distribución definidas en los requisitos de información. Cada requisito tiene tablas que guardan sus datos y consultas que lo responden, y cada regla del negocio está implementada en la base y no depende del programa cliente ni de que el usuario la recuerde.

La revisión del modelo físico antes de implementarlo fue decisiva: diez errores, entre ellos llaves foráneas que apuntaban a la propia tabla, habrían impedido registrar un pedido con varios productos. La combinación de restricciones declarativas (PK, FK, UNIQUE, CHECK) con triggers para las reglas de varios pasos dejó una base que resistió 62 de 62 intentos de romperla y aprobó 35 de 35 pruebas de aceptación.

El uso de inteligencia artificial para producir datos sintéticos aceleró el poblamiento, pero no reemplazó el criterio del equipo: la primera versión no respetaba el volumen acordado y una versión posterior dejó todos los lotes vencidos. Solo la validación con SQL permitió detectar y corregir esos problemas (Anexo 1).

8. Recomendaciones

- Conectar a los usuarios con los roles de `07_seguridad.sql`; reservar el superusuario para la administración.
- Programar respaldos automáticos (pg_dump diario o los respaldos de Supabase) y probar la restauración periódicamente.
- En una siguiente versión, agregar las entidades *Bodega* y *Movimiento de inventario* para registrar despachos y varias ubicaciones, con lo que la existencia dejaría de depender solo de los lotes vigentes.
- Registrar en la auditoría, además del usuario de base de datos, la estación desde la que se hizo cada cambio.
- Revisar los planes de ejecución (EXPLAIN ANALYZE) cuando el volumen crezca y crear índices compuestos para los reportes más usados.
- Mantener la validación SQL (08) como paso obligatorio antes de importar cualquier dato generado por IA.

9. Glosario

Término	Definición
Atomicidad	Propiedad de una transacción: se ejecuta completa o no se ejecuta nada.
Auditoría	Registro histórico de quién cambió qué dato y cuándo.
C4	Modelo para describir la arquitectura de software en cuatro niveles: contexto, contenedores, componentes y código.
Cardinalidad	Número de ocurrencias de una entidad que se relacionan con una ocurrencia de otra (uno, muchos).
CHECK	Restricción que obliga a que el valor de una columna cumpla una condición.
COPY	Instrucción de PostgreSQL para cargar o exportar datos desde un archivo, por ejemplo CSV.
CRUD	Operaciones básicas sobre los datos: crear, leer, actualizar y eliminar.
DDL	Lenguaje de definición de datos: CREATE, ALTER, DROP.
DML	Lenguaje de manipulación de datos: INSERT, UPDATE, DELETE.
Entidad	Objeto del negocio del que se guarda información, como Producto o Proveedor.
Llave foránea (FK)	Columna que guarda la llave primaria de otra tabla y crea la relación entre ambas.
Llave primaria (PK)	Columna que identifica cada fila sin repetirse.
Lote	Conjunto de unidades recibidas en una entrega, con una misma fecha de vencimiento.
Normalización	Proceso que organiza las tablas para eliminar la redundancia (1FN, 2FN, 3FN).
ON DELETE RESTRICT	Regla de una FK que impide borrar una fila que tiene filas relacionadas.
PL/pgSQL	Lenguaje procedimental de PostgreSQL con el que se escriben funciones y triggers.
Supabase	Plataforma que ofrece PostgreSQL administrado en la nube.
Trigger	Función que la base ejecuta automáticamente antes o después de una operación sobre una tabla.
UAT	Pruebas de aceptación de usuario: comprueban que el sistema cumple los requisitos antes de producción.
WAL	Registro de escritura anticipada; permite a PostgreSQL recuperarse tras una caída sin perder consistencia.

10. Bibliografía

American Psychological Association. (2020). *Publication manual of the American Psychological Association* (7.ª ed.). https://doi.org/10.1037/0000165-000

Anthropic. (2026). *Claude* (versión Opus 5.5) [Modelo de lenguaje de gran tamaño]. https://claude.ai/

Brown, S. (s. f.). *The C4 model for visualising software architecture*. https://c4model.com/

Chen, P. P.-S. (1976). The entity-relationship model: Toward a unified view of data. *ACM Transactions on Database Systems*, *1*(1), 9–36. https://doi.org/10.1145/320434.320440

Codd, E. F. (1970). A relational model of data for large shared data banks. *Communications of the ACM*, *13*(6), 377–387. https://doi.org/10.1145/362384.362685

Cuesta Quintero, B. (2026). *Bases de datos relacionales: modelo conceptual, modelo lógico, DDL y triggers* [Diapositivas y talleres de clase]. Universidad Francisco de Paula Santander Ocaña.

Elmasri, R. y Navathe, S. B. (2016). *Fundamentals of database systems* (7.ª ed.). Pearson.

PostgreSQL Global Development Group. (2024a). *PostgreSQL 16 documentation*. https://www.postgresql.org/docs/16/

PostgreSQL Global Development Group. (2024b). Constraints. En *PostgreSQL 16 documentation*. https://www.postgresql.org/docs/16/ddl-constraints.html

PostgreSQL Global Development Group. (2024c). Trigger functions. En *PostgreSQL 16 documentation*. https://www.postgresql.org/docs/16/plpgsql-trigger.html

PostgreSQL Global Development Group. (2024d). Reliability and the write-ahead log. En *PostgreSQL 16 documentation*. https://www.postgresql.org/docs/16/wal.html

PostgreSQL Global Development Group. (2024e). COPY. En *PostgreSQL 16 documentation*. https://www.postgresql.org/docs/16/sql-copy.html

Postobón S.A. (s. f.). *Postobón* [Sitio web]. https://www.postobon.com/

Silberschatz, A., Korth, H. F. y Sudarshan, S. (2020). *Database system concepts* (7.ª ed.). McGraw-Hill Education.

Supabase. (s. f.). *Supabase documentation*. https://supabase.com/docs

Anexo 1. Instrumento de recolección de evidencias

Este anexo documenta el uso de inteligencia artificial para producir los datos sintéticos que poblaron la base. **LLM utilizado:** Claude, versión Opus 5.5, de Anthropic (2026), a través de la aplicación web claude.ai. Los prompts que se presentan son los que el equipo escribió realmente en la conversación; se transcriben con corrección mínima de ortografía.

E1. Diseño relacional

El esquema de las tablas y sus restricciones se presenta en la sección 3.2 (diccionario de datos e instancias) y en la Figura 3 (modelo físico). Para la generación de datos, las restricciones que la IA debía respetar fueron:

Tabla 45

Restricciones entregadas a la IA para generar los datos

Tabla	Restricciones que debían cumplir los datos
product	Precio > 0; min_stock ≥ 0; max_stock > min_stock; unidad de una lista de 7 valores; nombre único sin importar mayúsculas; FK a categoría existente.
supplier	NIT único con formato #########-# y dígito de verificación; nombre y dirección de mínimo 3 y 5 caracteres.
employee	Cargo de una lista de 8; usuario único, en minúsculas, sin espacios ni tildes, de 4 a 30 caracteres.
supply_order	Nace PENDIENTE; fecha no futura; transiciones de estado válidas; total calculado por trigger y ≤ NUMERIC(12,2).
order_detail	Cantidad entre 1 y 100.000; precio > 0; un producto una sola vez por orden; solo en órdenes PENDIENTES.
batch	Solo para órdenes APROBADAS; vencimiento posterior a la orden y ≤ 5 años; cantidad inicial ≥ lo detallado.
batch_detail	Producto pedido en la orden; suma recibida ≤ cantidad pedida; suma del lote ≤ cantidad inicial.

E2. Prompt inicial

Fecha: 27/10/2026. **Prompt inicial (P1):**

```
Necesito hacer una base de datos para el inventario de Postobón, según la foto que te
mandé (modelo físico). Necesito que nos ayudes a crear bien la base de datos, todo acorde a
lo que pide el profesor, y necesito hacer un documento donde esté todo bien especificado de
lo que se hizo. Por ejemplo, si tuve un error en la creación de la tabla batch, tengo que
documentar qué error tenía y cómo lo solucioné. El profesor dijo que nuestra base de datos
la iba a tratar de reventar, entonces toca hacerla muy bien. A las tablas toca meterles
mucha información, según él más de 1000 filas. La eliminación no puede ser en cascada y hay
que aplicar lo que él nos dio en las clases.
```

Por qué se incluyeron esas instrucciones:

- *La foto del modelo físico* para que la IA generara datos con exactamente las tablas y columnas del diseño del equipo.
- *"Más de 1000 filas"* porque era la indicación del docente para demostrar que la base soporta volumen.
- *"La eliminación no puede ser en cascada"* y *"la va a tratar de reventar"* para que los datos se cargaran sin desactivar restricciones: si un dato es inválido, la carga debe fallar.
- *Los PDF de clase adjuntos* (DDL, esquemas, triggers) para que el código siguiera lo visto en la asignatura.

E3. Excel inicial

El primer intento se entrega como archivo adjunto **E3_excel_inicial_IA.xlsx** (una hoja por tabla y una hoja LEEME). Su volumen fue:

Tabla 46

Volumen del primer intento generado por la IA

Tabla	Filas	Tabla	Filas
category	20	supply_order	5.000
product	2.459	order_detail	27.194
supplier	1.000	batch	6.679
employee	1.000	batch_detail	23.685

Nota. En total 67.037 filas, generadas con un script de Python escrito por la IA y cargadas en PostgreSQL con los triggers activos.

E4. Registro de revisión

Cada versión de los datos se revisó con consultas SQL contra las restricciones de E1 y contra los acuerdos del equipo. Se registran los problemas encontrados y también las verificaciones que confirmaron que un criterio sí se cumplía:

Tabla 47

Registro de revisión de los datos generados

ID	Versión	Verificación	Hallazgo	Acción
R-01	Borrador	Revisión de una muestra aleatoria de filas	Nombres y marcas sin tildes ni Ñ (POSTOBON, PINA, CUCUTA).	Corregido antes de la v1: nombres con ortografía correcta.
R-02	Borrador	Muestra aleatoria de empleados	Nombres de género mezclado ("CARLOS	Listas separadas por género y vías

ID	Versión	Verificación	Hallazgo	Acción
		y proveedores	ALEJANDRA") y direcciones sin sentido ("KM 27 VÍA YUMBO ÁBREGO").	que salen de Ocaña.
R-03	Borrador	Carga contra el CHECK employee_username_ck	Usuarios con tilde (jpérez) violaban el formato a-z.	El generador quita tildes al construir el usuario.
R-04	v1	SELECT max(total) FROM supply_order	Una orden suma $115.242.663,20 y no cabe en NUMERIC(10,2): numeric field overflow.	TOTAL ampliado a NUMERIC(12,2) en el DDL.
R-05	v1	Conteo de filas por tabla	Volumen distinto al acordado por el equipo (5.000 órdenes, 2.459 productos).	Nuevo prompt P2 con el volumen exacto.
R-06	v1	Revisión de tablas base	La IA inventó 1.000 proveedores y 1.000 empleados; el equipo quería llenarlos.	P2: el script toma los ID que existan, por posición.
R-07	v2	Carga con solo 3 categorías (una borrada en medio), 2 proveedores y 1 empleado	Confirmado: la carga funciona con cualquier dato base del equipo.	Sin cambios.
R-08	v2	SELECT estado, count(*) FROM v_existencias	0 lotes vigentes: 100 productos "SIN EXISTENCIA"; RI-02 y RI-08 no se podían demostrar.	v3: fechas asignadas según el estado de la orden.
R-09	v3	08_validacion_datos.sql (V1 a V12)	Confirmado: 0 huérfanos, 0 totales descuadrados, 0 excesos, 196 lotes vigentes y 54 por vencer.	Aprobado para importar.
R-10	v3	04_pruebas_ataque.sql	Confirmado: 62 de 62 ataques bloqueados con los datos cargados.	Aprobado.

E5. Historial de interacción

Tabla 48

Historial de prompts y correcciones

#	Fecha	Prompt o instrucción	Resultado	¿Sirvió?
P1	27/10/2026	Prompt inicial (E2): base completa con más de 1000 filas, sin cascada, acorde a las clases.	v1 con 67.037 filas. Cumplía las reglas, pero el volumen y las tablas base no eran las acordadas (R-04 a R-06).	Parcialmente: sirvió la estructura; se rechazó el volumen.
P2	27/10/2026	"Hay que modificar los tamaños: order_detail debe tener 1500 filas, product 100, batch_detail 500, batch 500, supply_order 1000, y los otros nosotros los llenamos."	v2 con el volumen exacto; tablas base del equipo tomadas por posición; validada con datos mínimos (R-07).	Sí.
P3	05/11/2026	Dentro de la solicitud de completar este documento: revisar la coherencia de los datos con los requisitos y corregir lo que no cumpla.	La validación detectó 0 lotes vigentes (R-08); v3 asigna fechas según el estado: pendientes en los últimos 45 días, aprobadas en 120 días y 40 % de las recibidas en los últimos 10 meses.	Sí: 196 lotes vigentes y 54 por vencer.
P4	05/11/2026	Exportar los datos validados a Excel y CSV y preparar la importación con COPY, según el Anexo 1 de la rúbrica.	E6 (Excel definitivo), 8 CSV y 03b_importar_csv.sql, probado en una base vacía.	Sí.

Justificación de las modificaciones. P2 se pidió porque el volumen de la v1 no era el acordado por el equipo y porque las tablas base debían ser propias. P3 se pidió porque sin lotes vigentes la base

no podía demostrar los requisitos RI-02 y RI-08, aunque cada fila individual fuera válida: un dato puede pasar todas las restricciones y aun así no representar bien el negocio.

E6. Excel definitivo

El archivo validado antes de la importación se entrega como **E6_excel_definitivo.xlsx**: 8 hojas (category 20, supplier 50, employee 50, product 100, supply_order 1.000, order_detail 1.500, batch 500 y batch_detail 500 filas) y una hoja LEEME. Cada hoja tiene los nombres de columna del modelo físico; las fechas están en formato ISO (AAAA-MM-DD) y los montos sin separador de miles.

E7. Base de datos poblada

Los datos del Excel definitivo se exportaron a CSV y se importaron con COPY (PGDG, 2024e) mediante `03b_importar_csv.sql`. Como los ID son GENERATED ALWAYS y el trigger T2 exige que toda orden nazca PENDIENTE, el script usa tablas de paso y recorre el ciclo de vida de cada orden:

Tabla 49

Fragmento de 03b_importar_csv.sql

```
-- 1. Lectura del CSV en una tabla de paso
COPY s_supply_order FROM 'C:/Users/Public/postobon/csv/supply_order.csv'
     WITH (FORMAT csv, HEADER true, ENCODING 'UTF8');
-- 2. Las órdenes nacen PENDIENTES (regla del trigger T2)
INSERT INTO supply_order (order_id, supplier_id, issue_date) OVERRIDING SYSTEM VALUE
SELECT order_id, supplier_id, issue_date FROM s_supply_order ORDER BY order_id;
-- 3. Detalle -> APROBADA -> lotes -> RECIBIDA / CANCELADA
UPDATE supply_order o SET status = 'APROBADA'
  FROM s_supply_order s WHERE s.order_id = o.order_id AND s.status IN ('APROBADA', 'RECIBIDA');
```

Tabla 50

Resultado de la importación con COPY

Tabla	Filas en el CSV	Filas en PostgreSQL	Diferencias
product	100	100	0
supply_order	1.000	1.000	0 (estado y total)
order_detail	1.500	1.500	0
batch	500	500	0
batch_detail	500	500	0

Nota. Prueba realizada en una base vacía. La suma de los totales importados ($3.119.625.889,20) coincide con la de la base generada.

E8. Validación SQL

Las consultas de `08_validacion_datos.sql` comprueban la integridad de los datos importados.

Todas se ejecutaron sobre la base poblada con COPY:

Tabla 51

Consultas de validación y resultados

ID	Qué valida	Esperado	Obtenido	Interpretación
V1	Filas por tabla	100/1000/1500/500/500	100/1000/1500/500/500	Volumen acordado completo.
V2	Detalles huérfanos	0	0	Toda línea tiene orden y producto.
V3	Totales que no cuadran	0	0	El trigger T4 mantuvo los totales.
V4	Productos recibidos de más	0	0	Ningún lote supera lo pedido.
V5	Lotes en órdenes no aprobadas	0	0	Se respetó el ciclo de vida.
V6	Estados incoherentes	0	0	Recibidas con lotes; aprobadas con productos.
V7	Vencimientos fuera de rango	0	0	Todo vencimiento es posterior a la orden.
V8	Lotes excedidos	0	0	Ningún lote supera su cantidad inicial.
V9	Nombres duplicados	0	0	Catálogo sin repetidos.
V10	Fechas futuras	0	0 (04/01/2023 a 24/10/2026)	Historia de casi 4 años.
V11	Distribución de estados	370/250/190/190	370/250/190/190	Recibida, pendiente, aprobada, cancelada.
V12	Lotes vigentes y por vencer	> 0 y > 0	196 y 54	Permite demostrar RI-02 y RI-08.

E9. Reflexión individual

Dairo Julián Parada Reyes (0192288). Al principio pensé que con la IA los datos iban a salir listos y ya, pero no fue así. La primera vez no nos respetó las cantidades que habíamos pedido y en otra versión todos los lotes salieron vencidos, y eso no lo vimos a simple vista sino cuando hicimos las consultas de validación. Ahí entendí que la IA ayuda a ir más rápido, pero uno tiene que revisar todo lo que le entrega. También me pasó que los scripts que funcionaban en pgAdmin me dieron errores en Supabase, y arreglarlos me enseñó más de PostgreSQL que cualquier otra parte del proyecto.

Yiceth Yuliana Velásquez Gómez (0192312). Lo que más me costó fue entender por qué la base tenía que tener tantas reglas, porque al comienzo uno cree que con crear las tablas es suficiente. Cuando empezamos a intentar romperla con inserts y deletes raros me di cuenta de que sin los CHECK, las llaves foráneas y los triggers se podía guardar casi cualquier cosa. Con la IA aprendí que hay que explicarle muy bien lo que uno quiere, porque si el prompt queda incompleto los datos salen con errores. Me quedo con que revisar y probar es tan importante como diseñar.